

E.C.H.O.

Efficient Contextualized History Orchestrator

A local-first personal memory system unifying Gmail, Chrome, and YouTube into a single searchable retrieval pipeline.

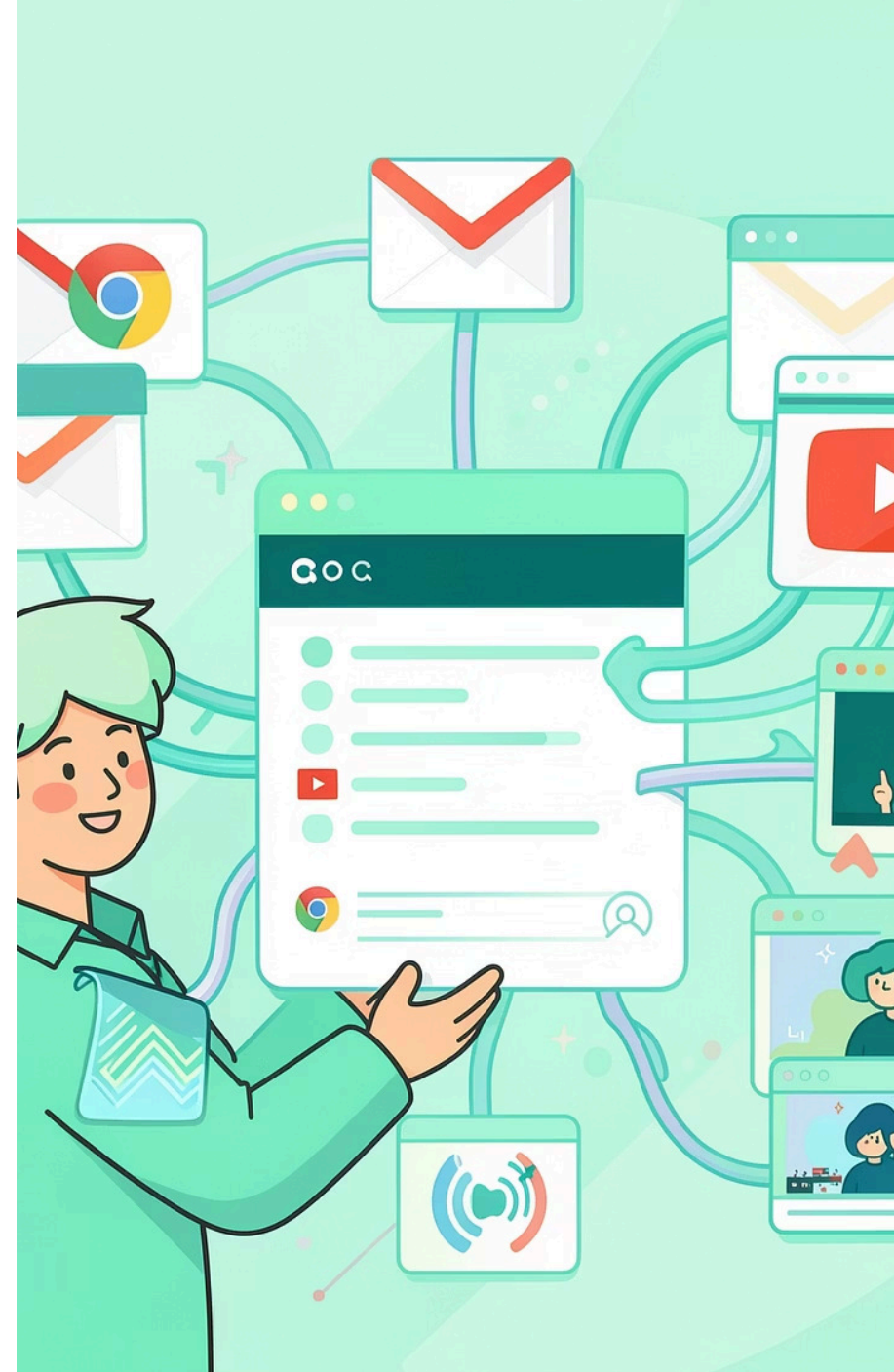
CAPSTONE PHASE II

Project Guide: Dr. M. Shalima Sulthana

Project ID : 92

Team Members:

- Arjun Sai N (PES2UG23CS091)
- Mohammed Mir Fazlai Ali (PES2UG23CS346)
- Mohammed Shehzaad Khan (PES2UG23CS349)
- Mrigank Jain (PES2UG23CS352)



Phase II establishes the system foundations: ingestion, intent-based filtering, a canonical PostgreSQL schema, storage engine refactoring, a retrieval skeleton, and enrichment groundwork for text cleaning, embeddings, FAISS interfaces, and system-group classification.

- ✔ The outcome is an integrated backend with source-specific metadata handling, Redis-backed revisit detection, unified storage abstractions, and early retrieval/enrichment execution paths.



Suggestions from Review - 1

Panel Appreciation & Feedback

Appreciation of Concept & Real-World Relevance

The panel acknowledged Echo as a unique and innovative idea addressing a genuine real-world problem — digital memory fragmentation across Gmail, Chrome, and YouTube. The cross-platform unification approach was appreciated for its originality and practical relevance.

Full Scope Implementation Commitment

The panel emphasized the importance of implementing the complete feature set outlined in Phase I scope, rather than delivering a reduced scope or prototype.

Response & Alignment

Architectural Commitment to Full Vision

We have structured Echo's system architecture modularly to ensure that all proposed features — Unified Search, Cross-Source Retrieval (RAG), Intent-Based Filtering, and Effort-Aware Ranking — are technically integrated and will be fully implemented across all three phases.



Suggestions from Review - 2

Guide Suggestions & Feedback

Suggestion 1: Presentation Scope Coverage

The presentation did not cover the full project scope and relied too heavily on verbal explanation.

Suggestion 2: LLM Fine-Tuning for Improved Results

Consider fine-tuning the LLM to improve results.

Response & Alignment

Response to Suggestion 1

Slides now explicitly cover the full scope of Echo, with every module, feature, and design decision shown on the slides.

Response to Suggestion 2

We did not fine-tune because small personal datasets often reduce generalization without improving personalization. The scope issue made the system seem simpler than it was; once the architecture was fully shown, the panel confirmed the complexity was appropriate. Instead, we improved results with LangGraph for stateful retrieval, memory, and deterministic execution.

E.C.H.O. at a Glance

The Solution:

Local-first personal memory system with intent-aware ingestion, semantic enrichment, and deterministic retrieval.

Phase II Status:

- ✓ **Complete:** Data ingestion (Gmail, Chrome, YouTube)
- ✓ **Complete:** Storage engine with atomic transactions
- ✓ **Foundation:** Enrichment pipeline (text cleaning, classification, embeddings)
- ✓ **Foundation:** Retrieval & synthesis engine (9-node LangGraph)
- ✓ **Exceeds** 20% implementation target

Key Metrics:

16 PostgreSQL tables with 19 indexes	384-dimensional FAISS vectors	9-node LangGraph with exactly 2 LLM calls per query
4-stage cascading classifier (~90% accuracy without LLM)	< 8 second end-to-end query response time	~450 MB storage per academic year
100% local-first (no cloud data storage)		

System Groups:

work, study, entertainment, personal, misc

Technology Highlights:

Python + FastAPI backend	PostgreSQL + FAISS + Redis storage
LangGraph + LangChain orchestration	Gemini 2.5 Flash LLM (provider-agnostic)
React + TailwindCSS frontend	Chrome Manifest V3 extension



Why E.C.H.O. Matters

Siloed Tools

Existing search is platform-locked, forcing users to remember where something happened before they can find it. Email, browser history, notes, and video consumption all live in separate systems, so context is lost across apps and related information is hard to connect.

Passive Behavior

Most personal knowledge tools assume active curation, but real user behaviour is passive, fragmented, and inconsistent. People rarely have time to tag, summarize, or reorganize every interaction, which means valuable signals remain buried across dozens of sessions and devices.

Hybrid Approach

Pure keyword search fails when users need cross-source reasoning or semantic matching across messy content. Full LLM systems can be costly, slow, and opaque, so E.C.H.O. combines deterministic retrieval, embeddings, and selective LLM use to balance accuracy, efficiency, and explainability.

Real-World Applications

Interview Preparation

E.C.H.O. can surface past notes, saved articles, and relevant conversations around a target company or role. It helps users quickly rebuild context and connect scattered preparation materials into a coherent study path.

Academic Workflows

Students and researchers can pull together lectures, papers, browser activity, and draft notes without manually organizing everything. This makes it easier to revisit sources, trace ideas, and keep track of what influenced a claim or argument.

Knowledge Continuity

When people return to an old project, E.C.H.O. helps reconstruct the original context from past interactions. That reduces rework and makes handoffs, resuming tasks, and long-running projects much easier to manage.

Wellbeing & Reflection

E.C.H.O. can help users reflect on habits, media consumption, and personal patterns over time. By connecting sessions across sources, it supports more thoughtful review without requiring constant manual journaling.

Analysis & Critique of Existing Systems:

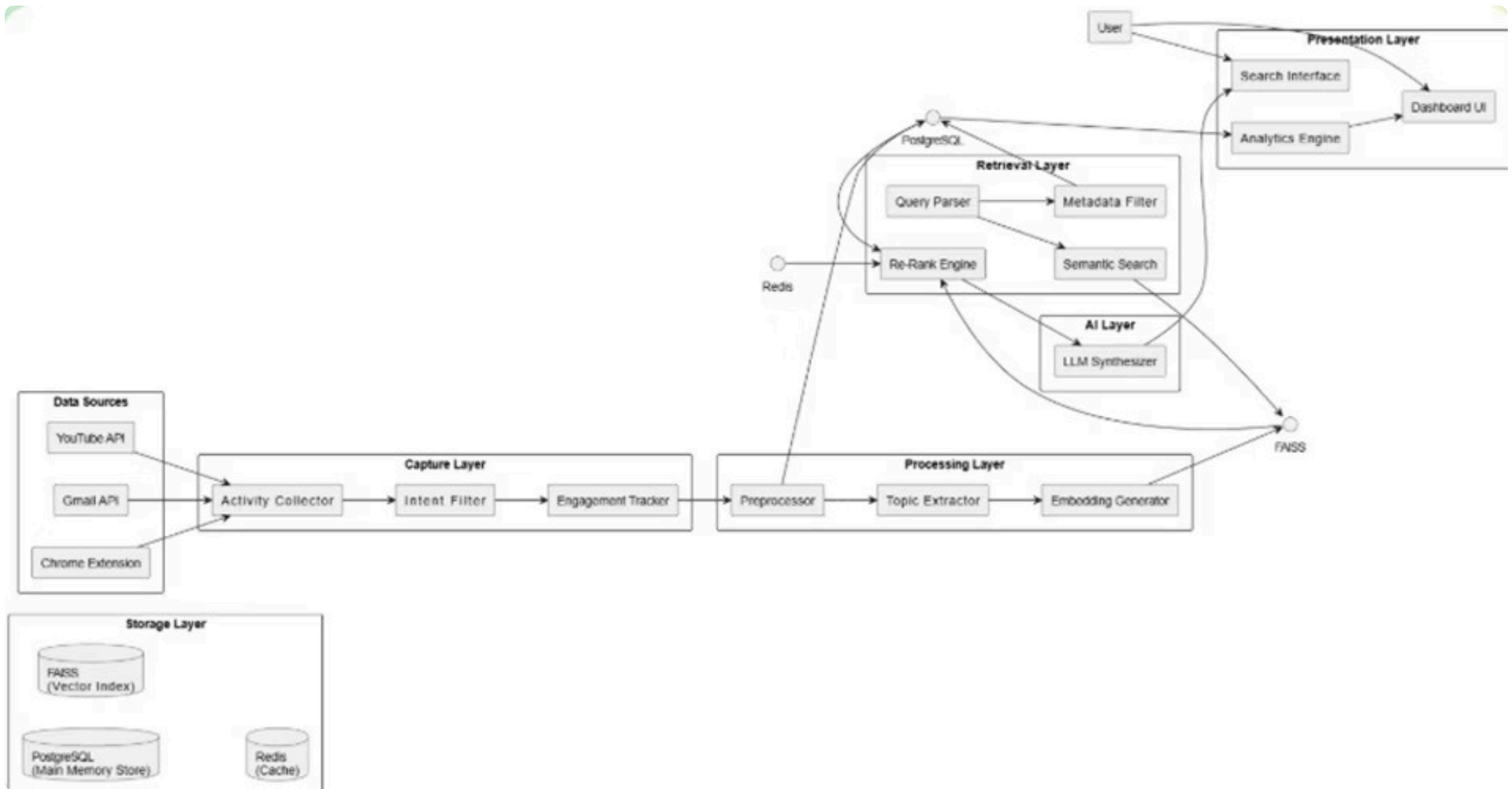
WEAKNESS OF EXISTING SYTEMS:

- Traditional keyword search tools are confined to single platforms (e.g., searching *only* Gmail) and fail to understand the semantic meaning or timeline of user actions.
- Current systems trap user data within proprietary boundaries. Your browser has no idea what you just discussed in an email, and your email client doesn't know what YouTube videos you watched to solve a work problem.
- Existing search engines struggle with chronological context. They cannot answer queries like *"find the article I read right before I watched that Python tutorial yesterday"* because they don't map events sequentially.

Relevance in Today's Landscape:

- **Aligning with Challenges:** Addresses the widespread issue of "Information Fragmentation," where users lose track of data scattered across emails, browsers, and videos.
- **Domain Impact:** By building a unified, multi-platform memory graph, this research directly advances the field of Personalized AI Assistants, moving from generic chatbots to deeply contextual, personal intelligence layers.

High-Level System Design

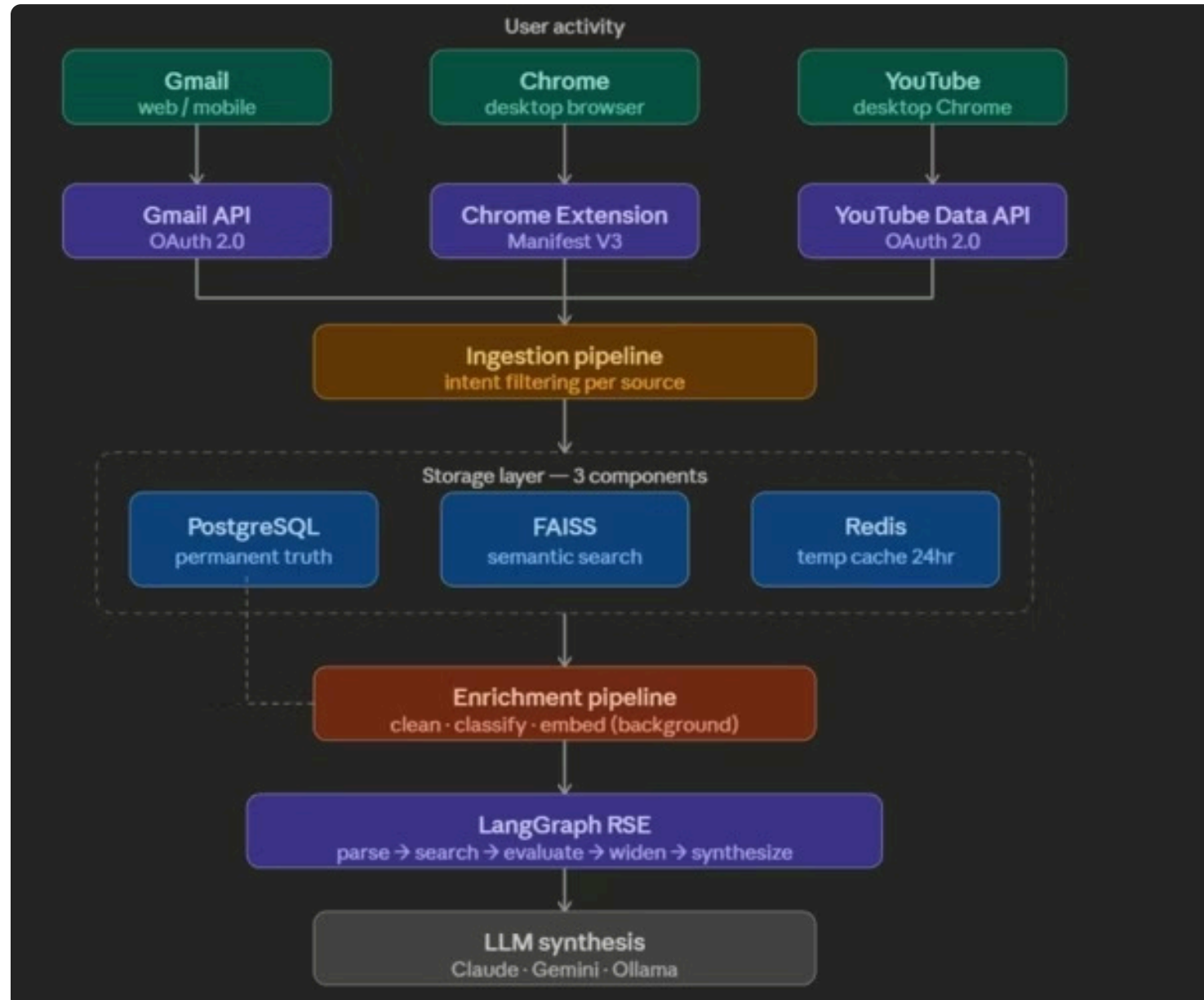


Interaction Flow

1. User activity captured by source-specific connectors
2. Intent-filtered items written atomically into canonical storage tables
3. Background enrichment prepares embeddable text and indexing metadata
4. Retrieval parses user query, searches memory items, returns contextual results
5. LLM synthesizes final answer with citations

i All writes route through a centralised storage engine — no ad hoc direct table access from connectors.

High Level Design



System Architecture Diagram

The layered architecture shows the complete data flow from capture through retrieval:

Data Flow Layers:

- **Data Sources:** Gmail API, Chrome Extension, YouTube Data API
- **Capture Layer:** Source-specific connectors with intent filtering
- **Processing Layer:** Enrichment pipeline (cleaning, classification, embedding)
- **Storage Layer:** PostgreSQL (permanent), FAISS (semantic index), Redis (cache)
- **Retrieval Layer:** LangGraph-based 9-node stateful graph
- **AI Layer:** LLM synthesis with Gemini 2.5 Flash
- **Presentation Layer:** React dashboard and Chrome Extension popup

Key Components:

- Gmail Connector (GMC): OAuth 2.0 polling with thread-level context
- Chrome Connector (CHC): Manifest V3 extension with two-phase intent gate
- YouTube Connector (YTC): URL detection + YouTube Data API v3
- Storage Engine (STE): Atomic transactions across all three storage layers
- Enrichment Pipeline (ENP): Background worker for text cleaning, classification, embedding
- Retrieval & Synthesis Engine (RSE): 9-node LangGraph with exactly 2 LLM calls per query

Design Methodology & Technology Stack



Backend

Python + FastAPI for API layer. Modular connector architecture reduces inter-module coupling.



Storage

PostgreSQL canonical schema. **Redis** for caching and revisit detection. **FAISS** semantic index interface.



Embeddings

sentence-transformers / **all-MiniLM-L6-v2** for lightweight local semantic embeddings without cloud dependency.



Orchestration

LangGraph / **LangChain** foundation for retrieval state graph and query routing. Chrome Extension **Manifest V3** for browser telemetry.

Design Approach

- Schema and architecture documents locked before module implementation began (design-first discipline)
- Modular source connectors built independently to reduce coupling
- Storage engine introduced to enforce atomic transaction patterns
- Enrichment and retrieval designed as independent background modules for phased integration
- Source-specific preprocessing and metadata-aware logic instead of generic pipeline
- Milestone-based implementation split across team members

❑ Schema and architecture documents were locked before any module implementation began — design-first discipline enforced throughout Phase II.

Data Sources & Preprocessing

Data Sources

E.C.H.O. captures live user activity in real-time from three sources:

- **Gmail API OAuth 2.0:** Email polling with thread-level context
- **Chrome Manifest V3 Extension:** Real-time engagement signals (foreground time, scroll depth, interaction count, revisit detection)
- **YouTube Data API v3:** Video metadata, transcripts (best-effort), Shorts classification

All data stored in PostgreSQL (Neon.tech during development, local at submission).

Data Model:

Unified representation: every email, web page, and YouTube video is a `memory_items` record, extended by source-specific metadata tables (`gmail_metadata`, `chrome_metadata`, `youtube_metadata`) and shared engagement table (`memory_engagement`).

Estimated Data Volume:

- Typical student user: ~1.25 MB/day
- 20 emails × 20 KB = 400 KB
- 35 web pages × 20 KB = 700 KB
- 10 YouTube videos × 15 KB = 150 KB
- Full academic year: ~450 MB (well within modern laptop storage)

Preprocessing Pipeline

All preprocessing done by Enrichment Pipeline as background worker (not on hot ingestion path).

Gmail Items:

- Strip HTML tags, remove tracking pixels
- Remove email signatures (pattern matching)
- Remove quoted reply content (lines starting with >)
- Result: `clean_body` with readable primary content
- Embeddable text: subject + clean body + thread keywords + sender domain hint

Chrome Items:

- Use Readability algorithm (Firefox Reader View)
- Extract main article content, strip navigation/sidebars/ads/footers
- Preserve headings
- Embeddable text: page title + extracted headings + content snippet + domain name

YouTube Items:

- Fetch auto-generated transcript (best-effort)
- Embeddable text: video title + description + channel name + category hint + first 500 chars of transcript

Embedding Generation:

- Model: all-MiniLM-L6-v2 (sentence-transformers)
- Output: 384-dimensional float32 vectors
- Indexed in FAISS
- Stored in `embedding_index` table with model version, vector dimension, active status
- `preprocessed` flag set to TRUE only after all steps complete successfully

Design Novelty & Innovation



Intent-Aware Ingestion + Deterministic Retrieval

Most personal data tools either capture everything (creating noise) or require manual curation (creating friction). E.C.H.O. uses per-source intent gates:

- Chrome: 2-phase gate requiring 5+ seconds exposure + engagement signals (scroll 25%+, 10+ seconds active time, interaction, or revisit)
- YouTube: 20+ seconds playback + manual interaction or revisit signal
- Gmail: No intent filter (all emails searchable for inbox awareness)

This automatically distinguishes genuine engagement from accidental exposure without user action.



LangGraph-Based Deterministic Retrieval

Instead of free-roaming agentic LLMs, E.C.H.O. uses a fixed 9-node stateful graph:

- LLM called exactly 2 times per query (intent parsing + answer synthesis)
- All retrieval logic between is deterministic Python
- Self-correcting evaluation loop widens search scope up to 3 times
- Every decision is visible, debuggable, and academically defensible
- Practical solution for sparse databases in early system usage



Four-Stage Cascading Classifier

System group assignment (work, study, entertainment, personal, misc) uses:

1. Structural API signals (~50% of items, zero cost)
2. Domain lookup for Chrome items (~25%)
3. Nearest centroid FAISS embedding comparison (~15%)
4. LLM batch fallback (~10%)

Achieves ~90% classification without LLM calls while maintaining accuracy through passive centroid improvement over time (monthly recomputation from confirmed items).

Technology Stack

Storage Layer

- PostgreSQL 16+
(Neon.tech → local)
- FAISS (faiss-cpu)
- Redis 7+ (Upstash)

Backend

- Python 3.10+
- FastAPI
- SQLAlchemy 2.x
- LangGraph

Enrichment

- sentence-transformers
(all-MiniLM-L6-v2)
- Readability (readability-lxml)
- PyPDF2 / pdfplumber
- python-docx
- pytesseract + Tesseract

Frontend

- React 18+ + TailwindCSS 3.x
- Recharts

Extension & LLM

- JavaScript (Manifest V3)
- Gemini 2.5 Flash (LangChain)

Key Features

- 16 PostgreSQL tables with 19 indexes
- 384-dimensional FAISS vectors
- 24-hour TTL for revisit signals
- 1-hour TTL for attachment cache
- Provider-agnostic LLM via LangChain (supports Claude, Gemini, Ollama)
- Local-first: no cloud services required after submission

Implementation Progress

1

Ingestion

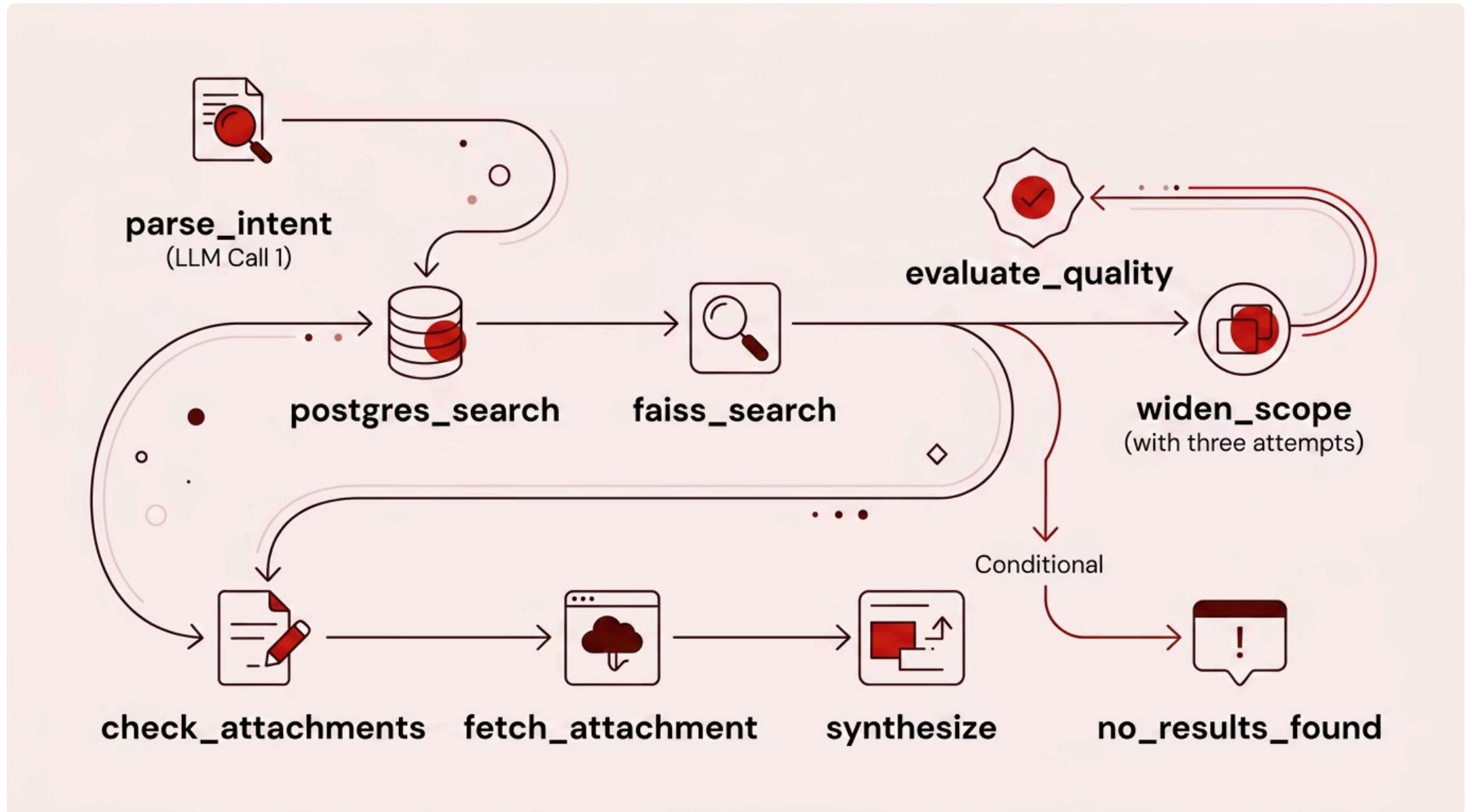
Gmail, Chrome, and YouTube connectors are implemented. Chrome Extension captures page and video interaction signals with intent filtering active, supporting source-level ingestion across browser activity and media engagement.

2

Storage

Canonical PostgreSQL schema and all source metadata tables are in place. The storage engine enforces atomic writes, and Redis revisit detection is operational for repeat-source awareness and durable persistence.

Retrieval & Synthesis Engine: 9-Node LangGraph



Key Features

- Exactly 2 LLM calls per query (intent parsing + synthesis)
- Multi-turn conversation memory via LangChain's PostgresChatMessageHistory
- Self-correcting evaluation loop for sparse databases
- All retrieval decisions visible and debuggable
- Full end-to-end query response time target: under 8 seconds

Implementation Progress — Retrieval & Enrichment

1

Retrieval (Foundation)

Retrieval endpoint and Postgres-first candidate search flow exist in working form. Query parsing and graph routing structure are defined, providing the base for semantic retrieval and downstream candidate ranking. The 9-node LangGraph structure is in place with conditional routing logic.

2

Enrichment (Foundation)

Text cleaner, embedding generator, FAISS manager, topic extractor, and system-group classifier are implemented in the enrichment branch. System groups are standardized to **work**, **study**, **entertainment**, **personal**, and **misc**. The four-stage cascading classifier is operational with structural signals, domain lookup, nearest centroid comparison, and LLM batch fallback.

Phase II Status:

Phase II target of minimum 20% implementation is exceeded at module-foundation level — ingestion, storage, and partial retrieval/enrichment are operational across main repo and teammate branches.

Next Steps:

Full end-to-end semantic maturity (production-grade FAISS integration, LLM synthesis) is scoped for Phase III.

Real-World Application

- **Use Case: "Train of Thought" Retrieval:** Users are no longer restricted to exact keywords. They can ask natural language questions like, *"Find that open-source repository I looked at right after reading my team lead's email last Tuesday,"* bridging the gap across different apps.
- **Use Case: Fact Verification & Source Tracing:** When writing a report and trying to remember where a fact came from, you can ask, *"Where did I see that statistic about database scaling?"* E.C.H.O instantly traces it back to the specific YouTube video timestamp or highlighted Chrome article snippet.
- **Use Case: Context-Aware Generation:** Beyond just finding links, the system synthesizes actual answers. It can draft a project update email by automatically pulling facts from YouTube tutorials watched and Chrome articles read yesterday.
- **Automated Progress Summaries (Standup Prep):** Instead of manually tracking what you did all week, you can ask, *"Generate a summary of my work-related research and communications from this week."* The system aggregates your sent emails, visited GitHub repos, and watched tutorials into a neat report.
- **Scalability & Cost-Efficiency:** To prevent API rate limits and high costs in the real world, the system uses batch processing. Ambiguous items are grouped up and processed together every few minutes, making it highly scalable as the user's personal data timeline grows.

Individual Contributions — Part 1



Mohammed Shehzaad Khan — Retrieval & Synthesis Engine

Defined retrieval state, graph routing structure, query parsing, and conversation-memory setup. Established Postgres-first candidate search flow for unified querying across memory sources.



Arjun — Gmail Ingestion & Classification

Implemented Gmail extraction and 4-stage system-group classification logic. Built metadata-aware classification using Gmail labels, YouTube categories, and Chrome domain signals to intelligently categorize content into 5 system groups (work, study, entertainment, personal, misc).

Individual Contributions — Part 2

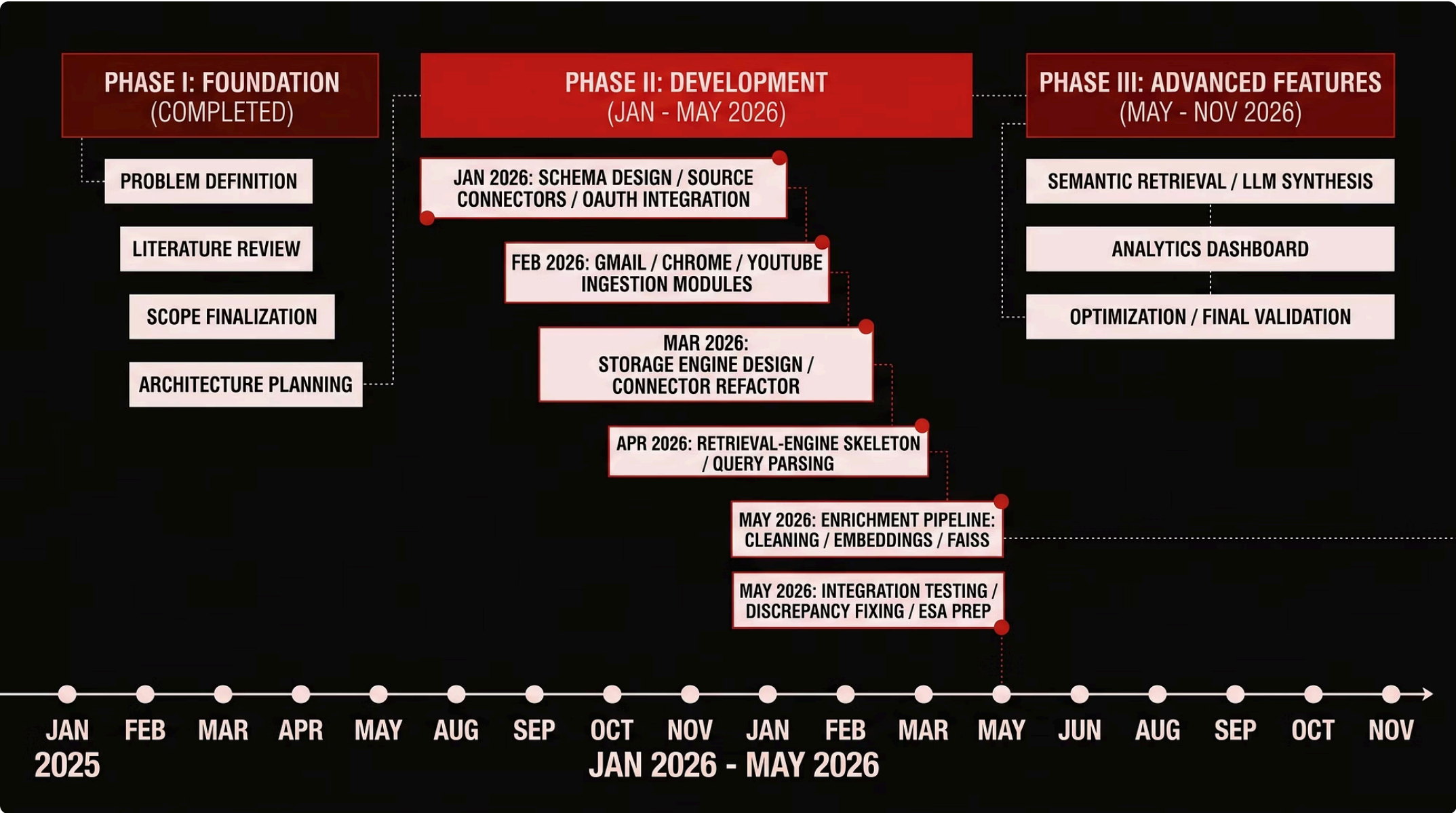
Mir — Enrichment Pipeline

● Implemented text cleaner, embedding generator, FAISS manager, topic extractor, and enrichment worker. Built source-specific embeddable text generation for Gmail, Chrome, and YouTube. Designed the four-stage cascading classifier with structural signals, domain lookup, nearest centroid comparison, and LLM batch fallback.

Mrigank — Storage Engine & Connector Refactor

● Implemented centralised storage engine, PostgreSQL manager, Redis manager, and FAISS interface stub. Refactored all connector storage paths toward atomic canonical writes and full schema alignment. Ensured all three connectors (Gmail, Chrome, YouTube) route through the unified `store_memory_item()` interface.

Project Timeline - Gantt Chart



The following outlines the project's key phases, milestones, and their respective timelines:

Phase I (Completed):

- Problem definition, literature review, scope finalization, architecture planning

Phase II (Current - Jan to May 2026):

- Week 1-2: Schema design, source connector setup, OAuth/API integration
- Week 3-4: Gmail/Chrome/YouTube ingestion modules
- Week 4-5: Storage engine design and connector refactor
- Week 5-6: Retrieval-engine skeleton and query parsing
- Week 6-7: Enrichment pipeline: text cleaning, embeddings, FAISS groundwork
- Week 7-8: Integration testing, discrepancy fixing, ESA preparation

Phase III (Future - May to Nov 2026):

- Full semantic retrieval, LLM synthesis, analytics dashboard, optimization, final validation

Project Timeline



Phase II Achievements

Architecture & Foundation:

- 16-table PostgreSQL schema deployed on Neon.tech with all constraints, indexes, foreign keys
- OAuth 2.0 authentication configured for Gmail and YouTube APIs
- GitHub repository with shared team access for parallel development

Data Ingestion (Complete):

- Gmail Connector: All emails via OAuth 2.0 polling with thread-level context enrichment
- Chrome Connector: Custom Manifest V3 extension with 2-phase intent gate (5s exposure + engagement signals)
- YouTube Connector: URL pattern detection, 20s playback threshold, metadata via YouTube Data API v3

Storage Engine (Complete):

- Centralized module with atomic transaction interface (Pattern A)
- UNIQUE(source_type, source_id) constraint prevents duplicates on restart
- All three connectors route through store_memory_item() interface

Enrichment Pipeline (Foundation):

- Text cleaning: HTML stripping, signature removal, quoted reply removal
- Four-stage cascading classifier: structural signals (50%) → domain lookup (25%) → centroid (15%) → LLM (10%)
- Embedding generation: all-MiniLM-L6-v2 producing 384-dim vectors
- FAISS indexing with embedding_index table for audit and rebuild

Retrieval & Synthesis Engine (Foundation):

- 9-node LangGraph stateful graph with exactly 2 LLM calls per query
- parse_intent: Structured JSON output from natural language
- postgres_search: Dynamic SQL across all 5 joined tables
- faiss_search: Filtered vector similarity search
- evaluate_quality: Deterministic 5-check assessment (no LLM)
- widen_scope: Progressive filter relaxation up to 3 times
- synthesize: LLM-generated cited answers with conversation memory

Phase III Roadmap

Phase 3 will complete the remaining implementation deliverables and stabilize the full Echo system.

1	Digital Wellbeing & Analytics Module (WBA) - Time aggregation queries for time spent per system category - Session computation identifying contiguous blocks of activity (5-minute gap threshold) - User groups feature with hybrid KNN plus rule classification engine - Human-in-the-loop approval for group suggestions - Per-item and per-session regret marking with pattern detection - LLM-generated weekly insights from aggregated numeric data only
2	React Dashboard & User Interface Layer (UIL) - Unified search interface with synthesized answer display - Source citation cards with timestamps and excerpts - Daily summary with system category time breakdown - Time heatmap grid with regret markers - Source breakdown analytics - User groups management interface with weekly review queue - Regret analytics charts - Settings panel for LLM API key configuration and source enable/disable - Chrome extension popup with quick search bar and recent items list
3	End-to-End Integration & Testing - Gmail ingestion tests - Chrome intent gate tests - YouTube intent gate tests - LangGraph single-turn and multi-turn query tests - Self-correcting loop tests - Performance benchmarks - Privacy audit to verify no personal data leaves local machine

Timeline: Target completion November 2026, with bug fixes and optimization through submission.

References

- [1] Y. Zhang, S. Liu, and J. Wang, "Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL," in Proc. IEEE Int. Conf. Data Eng. (ICDE), 2024, pp. 3640–3653.
- [2] R. Islambouli, S. Ingram, and D. Gillet, "Understanding Digital Wellbeing Through Smartphone Usage Intentions and Regrettable Patterns," in Proc. IEEE Int. Conf. Healthcare Informatics (ICHI), 2024.
- [3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP), 2019.
- [4] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.
- [5] M. Kleppmann et al., "Local-First Software: You Own Your Data, in Spite of the Cloud," in Proc. ACM SIGPLAN Symp. New Ideas, New Paradigms, Reflections Program. Softw. (Onward!), 2019, pp. 154–178.
- [6] V. Kostakos, A. Giannoulis, and M. Langheinrich, "Privacy-Preserving Data Storage and Processing: A Survey of Modern Approaches," ACM Comput. Surv., vol. 54, no. 6, 2021.
- [7] Y. Sun, A. Dubey, and E. P. Xing, "Lifelong Learning with Retrieval-Augmented Memory," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020.
- [8] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP), 2020.